
SENTINEL

Public Safety AI System

Complete System Architecture, Technology Stack, UI Specification & Implementation Roadmap

Version 1.0 | Confidential

System Name	SENTINEL — Real-Time Public Safety AI
Scope	Threat detection, live mapping, emergency dispatch
Primary Region	India (extensible globally)
Core Stack	YOLOv8 · FastAPI · React · PostgreSQL · Leaflet.js
Compliance	IT Act 2000, PDPB 2023, NDMA guidelines

SENTINEL is a multi-layer AI platform that ingests video feeds and open data sources, applies real-time computer vision and behavioural analytics, and surfaces actionable intelligence to human operators through a command dashboard, a live geospatial threat map, and a one-tap emergency dispatch interface. The system is designed for Indian public-safety infrastructure but is fully extensible to any jurisdiction.

Core Objectives

Detect	Identify threats — perimeter breaches, abandoned objects, crowd surges, loitering — from CCTV at real-time inference speed (<50 ms/frame).
Assess	Compute per-zone risk scores, rank alerts by severity, and suppress duplicates to prevent operator fatigue.
Communicate	Deliver a live threat map consumable by field officers and command centres, with self-updating incident timelines per region.
Dispatch	Enable one-touch escalation to the correct emergency authority (112, 181, 100, 108, 1098) based on the operator's geolocated region.
Learn	Continuously retrain models on confirmed-positive events to reduce false-positive rates over time.

SECTION 02 System Architecture

Layer-by-layer breakdown

SENTINEL is organised into five horizontal layers. Data flows upward from raw camera streams at the bottom to the operator interface at the top. Each layer is independently scalable and replaceable.

Layer 5 — Presentation

- React + Vite — Operator command dashboard
- Leaflet.js — Live threat map with GeoJSON overlays
- Browser Geolocation API — Live user positioning
- tel: protocol — One-tap emergency dispatch

Layer 4 — Application API

- FastAPI (Python) — REST + WebSocket endpoints
- Redis Pub/Sub — Real-time alert streaming
- Celery + Redis — Async task queue for heavy jobs
- JWT Auth — Role-based access for operators

Layer 3 — AI Inference Engine

- YOLOv8 (Ultralytics) — Object & person detection
- ByteTrack / DeepSORT — Multi-object tracking
- OSNet — Cross-camera re-identification
- Scikit-learn — Anomaly scoring & alert ranking

Layer 2 — Data Ingestion

- OpenCV — RTSP/ONVIF stream capture & frame pipeline
- FFmpeg — Video decoding & preprocessing
- ONNX Runtime / TensorRT — GPU-optimised inference
- Apache Kafka — High-throughput event streaming

Layer 1 — Data Sources

IP CCTV cameras (RTSP streams)

India NCRB / state crime open datasets

GADM GeoJSON — State & district boundaries

Nominatim (OSM) — Reverse geocoding

Arrows represent upward data flow: raw frames → inference events → ranked alerts → WebSocket push to frontend → operator action or auto-dispatch.

Screen 1 — Command Dashboard

The primary operator screen. It renders in real time via a WebSocket connection to the SENTINEL backend. The layout is split into two rows of cards.

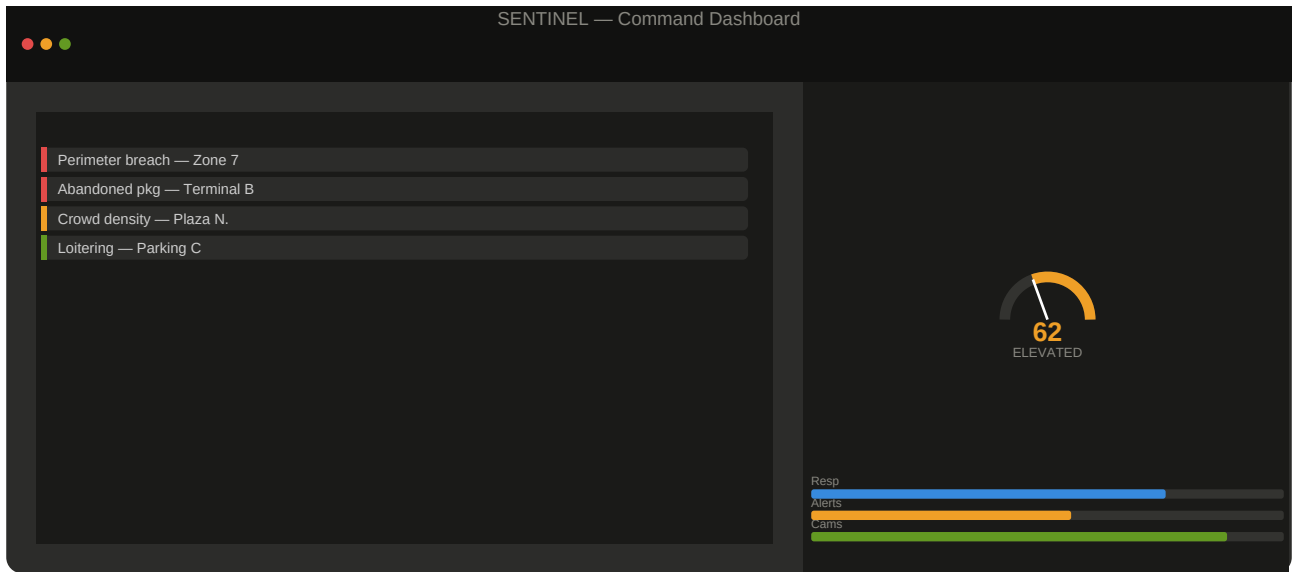


Fig 1. Command Dashboard — threat queue, risk dial, zone map, camera grid

Threat Queue: Left-panel card listing all active incidents ordered by severity (HIGH → MED → LOW). Each row shows a colour-coded severity badge, incident type, camera reference, and elapsed time. A 3px left border provides instant visual triage.

Risk Index Dial: A semicircular gauge computed as a weighted sum of active HIGH (×3), MED (×1.5), and LOW (×1) incidents divided by camera count. Animates every 1.8 s. Thresholds: <45 = Normal (green), 45–69 = Elevated (amber), ≥70 = High (red).

Stat Row: Three mini progress bars under the dial: camera uptime percentage, alerts per hour, and average response time in minutes. Sourced from the backend /metrics endpoint.

Zone Risk Map: 6-cell grid showing per-zone risk scores colour-coded to the same green/amber/red scheme. Clicking a zone deep-links to the live map filtered to that zone.

Camera Status Grid: 4-cell grid showing the most active cameras. ALERT cameras display an animated scan-line and a red border. Status dots pulse for cameras in detecting state.

Action Buttons: Escalate HIGH sends a POST to /dispatch/escalate and triggers a WhatsApp/SMS alert to the on-duty supervisor. Export Report POSTs to /reports/generate and returns a signed PDF download link.

Screen 2 — Live Threat Map

A full-page geospatial view built on Leaflet.js with OpenStreetMap tiles (no API key required). State polygons are rendered from GADM GeoJSON with a fill colour driven by the region's current risk level. Incident bubble markers are sized proportionally to incident count.

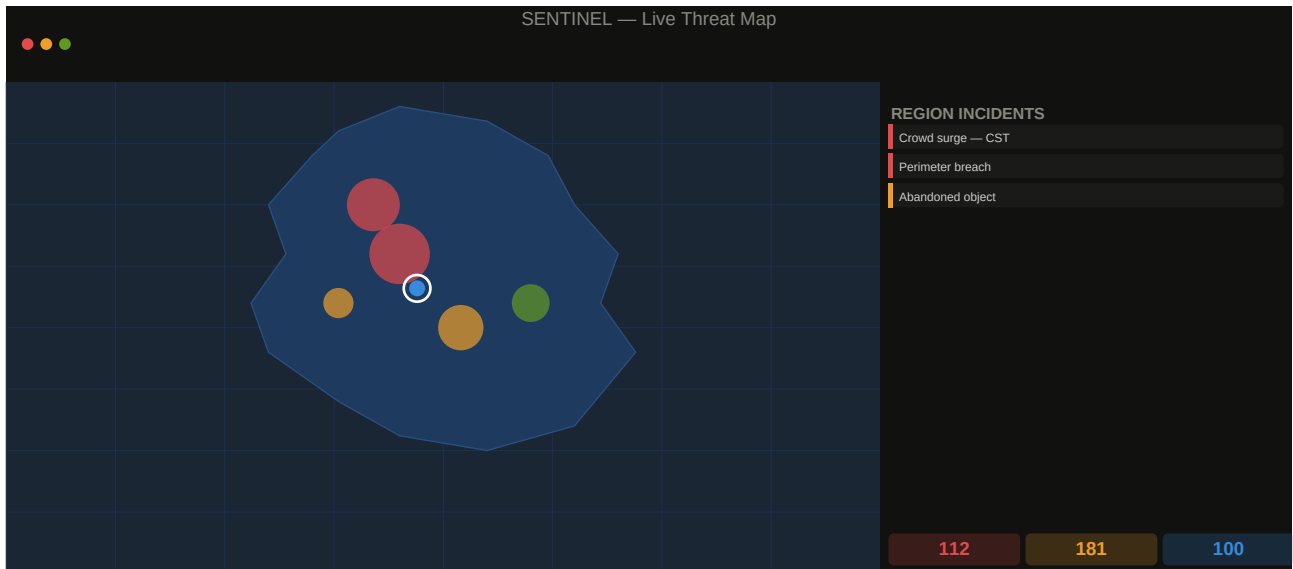


Fig 2. Live Threat Map — region polygons, incident bubbles, user pin, SOS panel

Region Click: Clicking a state polygon or incident bubble populates the right sidebar with all incidents for that region, fetched from `GET /incidents?region={name}`. The sidebar shows type, location, elapsed time, and severity badge per incident.

Live User Location: Browser Geolocation API fires on mount and every 30 s thereafter. User is shown as a blue pulsing circle marker. Reverse geocoding via Nominatim resolves the state name and auto-populates the emergency panel.

Emergency Panel: Six tap-to-call buttons rendered using the `tel:` URI scheme. Numbers resolve from a state-keyed lookup table: 112 (National), 181 (Women), 100 (Police), 108 (Ambulance), 1098 (Child), 1091 (Women Police). On mobile, tapping immediately initiates the call.

Polygon Colour Logic: RED fill = at least one HIGH severity incident. AMBER fill = MED incidents only. GREEN fill = LOW incidents only. GRAY fill = no incidents. fillOpacity scales with recency of last incident.

Screen 3 — Camera Analytics Grid

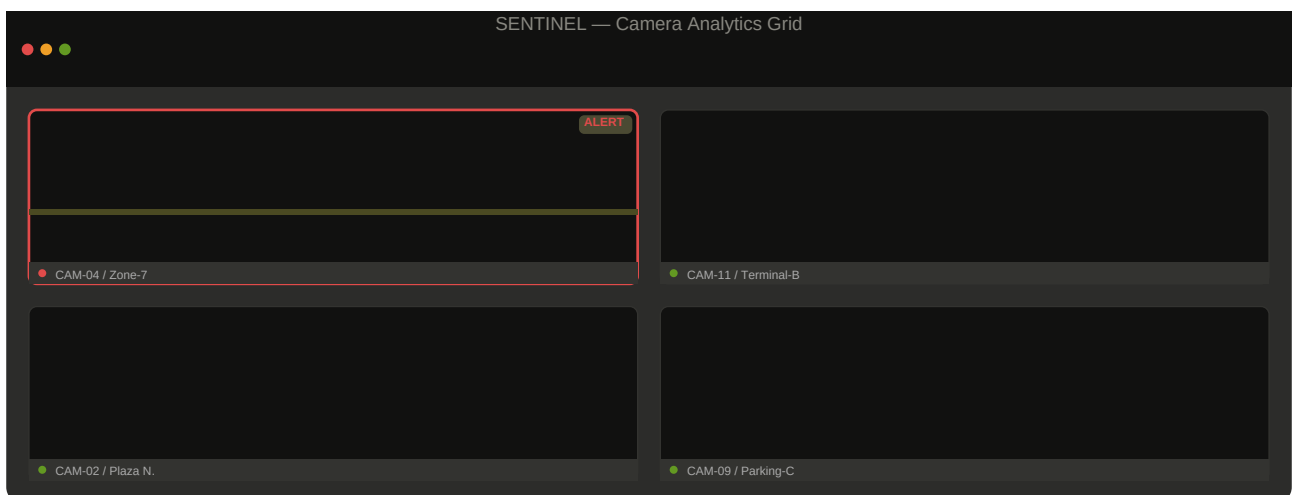


Fig 3. Camera Grid — live feed thumbnails with AI overlay annotations

Each cell streams a low-latency MJPEG thumbnail from the camera's RTSP feed via a backend proxy. The AI inference overlay draws bounding boxes, track IDs, and dwell-time counters directly on the frame using OpenCV before encoding. Alert cameras pulse a red border and display a scan-line animation. Clicking any cell opens a

full-screen modal with the raw detection event log and confidence history chart.

AI / ML

YOLOv8 (Ultralytics)

ByteTrack

DeepSORT

OSNet (torchreid)

OpenCV

ONNX Runtime

TensorRT

Scikit-learn

Pandas

NumPy

Backend

Python 3.11

FastAPI

Celery

Redis

Apache Kafka

FFmpeg

Pydantic

SQLAlchemy

Frontend

React 18

Vite

Leaflet.js

Chart.js

Tailwind CSS

Socket.IO Client

Share Tech Mono

Syne

Database

PostgreSQL 16

PostGIS

Redis (cache)

MinIO (video store)

DevOps / Infra

Docker

Docker Compose

NGINX

Prometheus

Grafana

GitHub Actions

Uvicorn

Gunicorn

Data Sources

GADM GeoJSON

OpenStreetMap / Nominatim

NCRB Open Data

NDMA API

GNews API

OpenWeatherMap

Technology Justification

YOLOv8	Object detection	State-of-art accuracy at 80+ FPS; Apache 2.0 licence; no cost
ByteTrack	Multi-object tracking	Surpasses SORT/DeepSORT; works without re-ID for speed
FastAPI	API layer	Async-first; native WebSocket; auto OpenAPI docs
Redis	Event stream + cache	Sub-millisecond pub/sub; ideal for alert fan-out

PostgreSQL + PostGIS	Incident store	Spatial queries (ST_Within) for region filtering
Leaflet.js	Map rendering	100% open source; OSM tiles; no API key or billing
React 18	Dashboard UI	Concurrent rendering; large ecosystem; ideal for real-time UIs
Docker	Deployment	Reproducible across dev / staging / prod; easy scaling

01	Frame Capture	OpenCV reads RTSP stream at 25 FPS per camera. FFmpeg decodes H.264. Frames buffered in ring buffer (128 frames).
02	Pre-processing	Frame resized to 640×640 (YOLOv8 input). Normalised to [0,1]. Batched by camera group (4 frames/batch).
03	YOLOv8 Inference	TensorRT-optimised engine runs inference. Outputs bounding boxes, class IDs, confidence scores. <15 ms/frame on T4 GPU.
04	Object Tracking	ByteTrack associates detections across frames. Assigns persistent track IDs. Computes velocity and dwell-time per track.
05	Behavioural Analytics	Rules engine evaluates: dwell_time > 600s → loitering. zone_count > threshold → crowd surge. Unowned object > 300s → abandoned object.
06	Alert Scoring	Scikit-learn rule ensemble assigns severity (HIGH/MED/LOW). Duplicate suppression via MinHash on (zone, type, time_window).
07	Event Publishing	Confirmed alerts published to Redis channel sentinel:alerts. Kafka topic sentinel.events stores full audit log.
08	API Delivery	FastAPI WebSocket endpoint pushes to all subscribed dashboard clients. REST endpoint stores alert in PostgreSQL with PostGIS geometry.
09	Frontend Render	React updates threat queue, risk dial, zone map. Leaflet bubble markers update colour & radius. Feed log prepends new entry.

Phase 1 — Foundation

Weeks 1–4

- Set up Docker Compose stack: FastAPI + Redis + PostgreSQL + PostGIS
- Implement OpenCV RTSP ingestion and frame pipeline
- Integrate YOLOv8 pretrained weights; validate on sample CCTV footage
- Build basic REST API: GET /incidents, POST /alerts, GET /metrics
- Deploy React shell with placeholder dashboard components

Phase 2 — Intelligence

Weeks 5–10

- Add ByteTrack multi-object tracking across frames
- Implement behavioural rules engine (loitering, crowd surge, abandoned object)
- Build alert scoring, deduplication, and severity ranking
- Kafka integration for full audit event log
- Dashboard WebSocket live updates; risk dial animation

Phase 3 — Geospatial

Weeks 11–15

- Integrate GADM GeoJSON state/district polygons into Leaflet map
- Build /incidents?region= PostGIS spatial query endpoint
- Browser Geolocation + Nominatim reverse geocoding
- Dynamic emergency helpline panel keyed by detected state
- tel: dispatch integration and call logging

Phase 4 — Hardening

Weeks 16–20

- TensorRT model optimisation for production GPU inference
- Prometheus + Grafana monitoring; model drift alerts
- Role-based access control (admin, operator, viewer)
- Load testing: simulate 25 concurrent cameras at 25 FPS
- Compliance review: PDPB 2023 data minimisation, audit trails

SENTINEL's ultimate objective is to shift public safety infrastructure from reactive to predictive — reducing the window between a threat emerging and a trained officer responding, without compromising civil liberties or requiring proprietary closed-source AI.

Sub-60-second detection-to-dispatch

From a camera flagging an anomaly to the nearest unit receiving a dispatch notification, the end-to-end latency target is under 60 seconds at full production load (25 cameras, 25 FPS each).

90%+ alert precision

A false positive rate above 15% causes operator fatigue and system abandonment. SENTINEL's behavioural rules engine, combined with confidence thresholding and deduplication, targets <10% false positives on loitering and crowd surge detection by Phase 4.

Zero dependency on facial recognition

The system is designed to be fully functional without biometric identification. This is both an ethical commitment and a legal necessity under India's PDPB 2023. All detection is behaviour-based and object-based, not identity-based.

Accessible emergency dispatch for every citizen

The SOS panel in the live map gives any user — not just operators — one-tap access to the correct authority for their location. For women in India, this means 181 is one tap away from wherever they are.

Open, auditable, deployable

Every model weight, every data source, and every line of infrastructure code is open-source or publicly licensed. The system can be deployed by any state government or NGO without vendor lock-in or licensing fees.

Continuous learning loop

Confirmed positive alerts feed back into a fine-tuning pipeline that retrains YOLOv8 on domain-specific footage monthly. The system improves with every deployment, not just with software updates.

SENTINEL is infrastructure for a safer public space — not a surveillance state. Its design philosophy holds that technology should augment human judgment, not replace it. Every alert requires a human decision.

Every dispatch is logged and auditable. Every citizen retains the right to walk freely without being identified.

SECTION 08 Core API Reference

Key endpoints

GET	/incidents	?region=&severity;=&limit;=	Returns paginated incident list with PostGIS spatial filter
POST	/alerts	Body: {camera_id, type, bbox, confidence}	Ingest new alert from inference engine
GET	/metrics	?window=1h	System metrics: uptime, alerts/hr, avg response time
GET	/zones		All zone risk scores with geometry centroids
WS	/ws/alerts	Bearer token	WebSocket stream of live alert events
POST	/dispatch/escalate	Body: {alert_ids, unit_id}	Escalate HIGH alerts to supervisor
GET	/reports/generate	?from=&to;=	Generate signed PDF incident report
GET	/cameras	?zone=	Camera list with status and last-seen timestamp

SENTINEL System Architecture Document — v1.0 — All technologies referenced are open-source or publicly licensed.